Spaceship Titanic Classification Through Structured Feature Engineering and Comparative Tree-Based Modeling

Lu Kejie (2430034044), Hou Rui (2430034019), Yang Zhongxing (2430034088), Guo Yitao (2430034015), Chen Ziheng (2430034008)

***Abstract*—This report presents a full machine learning workflow for the Kaggle Spaceship Titanic classification task. The goal of the task is to predict whether a passenger was transported to an alternate dimension using structured passenger information, including travel status, cabin location, demographic fields, and onboard spending behavior. Instead of treating the task as a simple model-fitting exercise, we built the project around data understanding, feature construction, model-specific preprocessing, and reproducible comparison. Exploratory data analysis showed that several raw columns contain strong hidden structure: PassengerId contains group information, Cabin contains spatial information, spending columns reflect passenger activity, and CryoSleep is closely connected with both the target and spending behavior. Based on these findings, we constructed group features, cabin location features, spending consistency features, surname-based family features, and interaction features. Five model routes were implemented and compared: CatBoost, XGBoost with KMeansSMOTE, LightGBM, ExtraTrees, and HistGradientBoostingClassifier. Each route was designed with a preprocessing pipeline suitable for its modeling assumptions. The final selected model was XGBoost with KMeansSMOTE, which achieved the strongest reproducible public Kaggle score of 0.81295. Validation diagnostics, including confusion matrix, ROC curve, and feature importance, were used to interpret the final model. The project shows that, for this tabular classification task, careful feature engineering and model-compatible preprocessing were more important than simply increasing model complexity.**

***Index Terms*—Binary classification, CatBoost, feature engineering, Kaggle, KMeansSMOTE, machine learning, tabular data, XGBoost.**

I. Introduction

The Spaceship Titanic competition is a binary classification task hosted on Kaggle. Each row of the dataset represents one passenger aboard the fictional Spaceship Titanic. The target variable, *Transported*, indicates whether the passenger was transported to an alternate dimension. The input features include passenger identity, home planet, cryo sleep status, cabin information, destination, age, VIP status, several spending columns, and passenger name. Although the story setting is fictional, the prediction problem is a realistic tabular machine learning task: the data are mixed-type, partially missing, and contain both direct variables and semi-structured string fields.

A naive way to approach the task would be to encode all categorical variables, impute missing values, train a classifier, and submit predictions. However, this would ignore much of the structure hidden in the raw columns. For example, *PassengerId* is not merely an arbitrary identifier; its prefix indicates passenger groups. *Cabin* is a string, but it contains deck, cabin number, and side information. *Name* is not directly numerical, but surname patterns may represent family-level relationships. The spending columns are sparse and skewed, yet they provide information about passenger activity and are strongly connected with *CryoSleep*. Therefore, the central problem in this project was not only selecting a strong model, but also designing a meaningful representation of the data.

The project followed a complete machine learning workflow. We first performed exploratory data analysis (EDA) to understand the target distribution, missing values, categorical patterns, cabin structure, and spending behavior. We then designed feature engineering rules based on those observations. After that, we implemented five different model routes: CatBoost, XGBoost with KMeansSMOTE, LightGBM, ExtraTrees, and HistGradientBoostingClassifier (HGBC). These models were compared using Kaggle public leaderboard scores and additional validation diagnostics. The final selected model was XGBoost with KMeansSMOTE, which achieved a public Kaggle score of 0.81295.

One practical decision in our workflow was to avoid forcing every model into the same preprocessing pipeline. Different algorithms have different requirements. CatBoost is designed to work with categorical variables directly, while XGBoost, LightGBM, ExtraTrees, and HGBC require encoded or otherwise numerical inputs. Treating every model as if it had the same input assumptions would make the comparison convenient but less fair. Therefore, we used a shared feature engineering philosophy while allowing each route to use preprocessing steps appropriate to the model.

The report is organized as follows. Section II reviews related techniques for tabular classification, tree-based models, categorical handling, oversampling, and validation. Section III formulates the dataset and prediction problem. Section IV presents the EDA findings and explains how they motivated feature engineering. Section V describes preprocessing and feature construction. Section VI explains the five model routes and their training procedures. Section VII reports experimental results and model analysis. Section VIII records the final Kaggle submission materials, including the public score, GitHub repository link, and the reason why the final ranking is not reported. Section IX discusses limitations. Section X presents future work. Section XI concludes the project. A contribution

All authors are members of the AI3023 Machine Learning Workshop course project team milkloong@MLW.

statement and appendices provide additional reproducibility details.

The main contributions of this project are as follows:

- We analyzed the dataset from multiple perspectives, including target balance, missingness, categorical effects, cabin location, and spending behavior.
- We transformed raw identifiers and semi-structured strings into structured features, including group, cabin, spending, family, and interaction features.
- We implemented five model routes and compared them under model-specific preprocessing assumptions.
- We selected the final model based on public score, reproducibility, pipeline clarity, and practical submission reliability.
- We interpreted the final model using validation accuracy, confusion matrix, ROC curve, and feature importance.

II. Related Work and Existing Techniques

A. Tabular Classification

Tabular classification is one of the most common forms of applied machine learning. Unlike image or text data, tabular data often consist of heterogeneous columns with different meanings, missing value patterns, categorical cardinalities, and numerical scales. This makes preprocessing and feature engineering particularly important. In many tabular projects, the final performance is not determined by the model alone. It also depends on whether the input representation exposes the useful structure of the dataset.

In the Spaceship Titanic task, several raw fields are not directly useful without transformation. PassengerId, Cabin, and Name are stored as strings, but each contains interpretable structure. Spending variables are numerical, but they are highly skewed and contain many zeros. Categorical variables such as HomePlanet, Destination, CryoSleep, and VIP have different relationships with the target. These characteristics are typical of real tabular datasets: the values are already organized in rows and columns, but the most predictive signals are not always directly accessible to a model.

B. Tree-Based Ensemble Models

Tree-based ensemble models are widely used for tabular prediction because they can capture nonlinear relationships and feature interactions. A single decision tree is usually unstable, but combining many trees can improve generalization. Random Forests and ExtraTrees aggregate multiple independently trained trees and reduce variance through ensemble averaging [2], [3]. Gradient boosting methods, such as XGBoost, LightGBM, CatBoost, and HGBC, build trees sequentially and focus on errors made by previous trees [4]–[7].

XGBoost is known for its strong performance on structured data. It combines gradient boosting with regularization, shrinkage, column subsampling, and efficient split finding [5]. LightGBM improves efficiency by using histogram-based algorithms and leaf-wise tree growth [6]. CatBoost focuses on categorical feature handling and ordered boosting, which is useful when the dataset contains many categorical columns [7]. HGBC provides a scikit-learn implementation of histogram-based gradient boosting and is useful as a reproducible baseline.

C. Categorical Feature Handling

Handling categorical variables is a central issue in this project. One-hot encoding is simple and avoids artificial ordering, but it can increase dimensionality and may not handle high-cardinality variables efficiently. Ordinal encoding is compact, but it may introduce false order relationships among categories. CatBoost uses specialized categorical handling techniques that allow it to work more naturally with categorical-heavy datasets.

For this reason, a single preprocessing pipeline is not necessarily fair to all models. A preprocessing strategy that helps XGBoost may not be optimal for CatBoost, and a compact ordinal encoding strategy for HGBC may not preserve categorical structure as effectively as CatBoost. Our project therefore adopted model-specific preprocessing. This choice made the comparison more realistic because each model was evaluated under conditions closer to its intended usage.

D. Synthetic Oversampling and KMeansSMOTE

SMOTE is a synthetic oversampling technique that creates new minority samples by interpolating between existing samples in feature space [8]. KMeansSMOTE extends this idea by first clustering the data and then applying oversampling in appropriate local regions. The motivation is that a dataset may not be globally imbalanced but can still contain local subgroup imbalance.

In the Spaceship Titanic dataset, the target distribution is relatively balanced overall. However, once passengers are divided by CryoSleep, HomePlanet, cabin deck, and spending behavior, local class distributions may differ. A model trained only on the global distribution may underrepresent some local patterns. KMeansSMOTE was therefore considered a reasonable addition to the XGBoost route because it matches the subgroup nature revealed by EDA.

E. Validation, Leaderboard Scores, and Reproducibility

Kaggle competitions provide public leaderboard scores based on hidden labels. These scores are useful because they evaluate submissions using a standardized metric. However, leaderboard performance alone is not enough for a course project. A model should also be reproducible, interpretable, and supported by local validation evidence. A one-time high public score can be misleading if the pipeline cannot be rerun or if the method depends on manual steps that are not documented.

Therefore, this project used both Kaggle public scores and local validation diagnostics. The public score was used for final model comparison because it is the official external metric. The confusion matrix, ROC curve, and feature importance were used to interpret model behavior on a held-out validation split. These diagnostics do not replace the Kaggle score, but they help explain how the final model makes predictions and whether it behaves reasonably on both classes.

TABLE I
RAW FEATURE GROUPS AND MODELING MEANING

Feature Group	Raw Columns	Modeling Meaning
Identity	PassengerId, Name	Passenger group and family structure
Travel status	HomePlanet, CryoSleep, Destination, VIP, Age	Passenger background and travel condition
Cabin location	Cabin	Deck, side, and approximate spatial location
Spending behavior	RoomService, FoodCourt, ShoppingMall, Spa, VRDeck	Activity level and consistency with CryoSleep
Target	Transported	Binary label for classification

III. DATASET AND PROBLEM FORMULATION

A. Competition Task

The project uses the Kaggle Spaceship Titanic dataset [1]. The task is to predict a Boolean target for each test passenger. A prediction file must contain the passenger identifier and a predicted value for *Transported*. The official evaluation metric is classification accuracy. Let $y_i \in \{0, 1\}$ be the true label of passenger i, and let $\hat{y}_i \in \{0, 1\}$ be the predicted label. Accuracy is defined as

$$\text{Accuracy} = \frac{1}{n} \sum_{i=1}^{n} \mathbb{I}(y_i = \hat{y}_i), \tag{1}$$

where n is the number of evaluated passengers and $\mathbb{I}$ is the indicator function.

Although accuracy is the official metric, it does not describe all aspects of model behavior. For this reason, we also examined the confusion matrix and ROC curve on a validation split. These local diagnostics helped us understand false positives, false negatives, and probability ranking quality.

B. Raw Feature Groups

The raw features can be grouped into four broad categories. First, passenger identity fields include PassengerId and Name. These fields do not directly represent numerical properties, but they can be transformed into group and family features. Second, travel-status features include HomePlanet, CryoSleep, Destination, Age, and VIP. These fields describe passenger background and travel condition. Third, cabin information is contained in the Cabin field, which combines deck, cabin number, and side. Fourth, spending behavior is represented by RoomService, FoodCourt, ShoppingMall, Spa, and VRDeck.

Table I summarizes the main raw fields and their role in the project. This table is included because the report should make clear that our feature engineering was not arbitrary. Each group of engineered features was designed from specific raw columns.

C. Project Constraints

The course project required a full workflow rather than a single notebook result. The final report needed to include an abstract, introduction, related work, methodology, experimental study and results analysis, future work and conclusion, references, contribution statement, final Kaggle ranking, GitHub link, and Kaggle submission log screenshots. The code also needed to be runnable and documented. These requirements affected the final model choice. A model route with a slightly higher one-time score but poor reproducibility would not be appropriate for final submission. We therefore considered reproducibility and clarity together with public score.

IV. EXPLORATORY DATA ANALYSIS

A. Purpose of EDA

EDA was used as a design step rather than a decorative part of the report. The purpose was to identify which raw columns contained meaningful signals and how those signals should be transformed. For this dataset, EDA helped answer several questions. Is the target distribution balanced enough for accuracy to be meaningful? Which categorical variables are strongly associated with the target? Should spending columns be combined? Is Cabin useful, or should it be discarded as a noisy string? Does the data suggest a global imbalance problem or a more local subgroup problem?

The answers to these questions directly influenced feature engineering and model selection. For example, the strong relationship between CryoSleep and Transported motivated CryoSleep-related interaction features. The variation across cabin decks motivated splitting the Cabin column. The spending distribution motivated TotalSpend, NoSpend, and LogTotalSpend. The difference across HomePlanet categories motivated preserving origin information and combining it with deck features.

B. Target Distribution

The target classes in the training data are relatively balanced. This is important because the official metric is accuracy. If one class dominated the dataset, accuracy could be misleading because a model could obtain a high score by predicting the majority class. In this dataset, however, both classes are represented at similar levels. Therefore, accuracy is acceptable as the main metric, while additional diagnostics are still useful for understanding model behavior.

The balanced target distribution also shaped our interpretation of KMeansSMOTE. We did not treat the entire dataset as a severe global imbalance problem. Instead, we considered that local imbalance may occur inside subgroups. A subgroup defined by CryoSleep, HomePlanet, deck, and spending behavior may have a different class distribution even if the whole dataset is balanced. This distinction is important because it explains why a local resampling method can still be useful.

C. CryoSleep Effect

CryoSleep is one of the strongest predictors in the dataset. Fig. 1 shows the transported rate by CryoSleep status. Passengers with CryoSleep enabled have a transported rate of about 0.82, while passengers without CryoSleep have a transported rate of about 0.33. This large gap suggests that CryoSleep

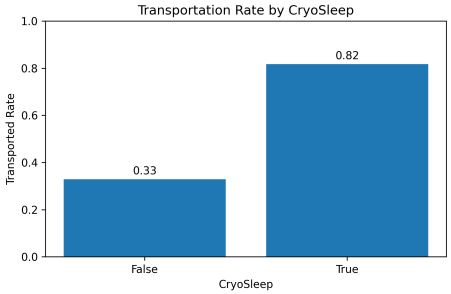

Fig. 1. Transportation rate by CryoSleep status.

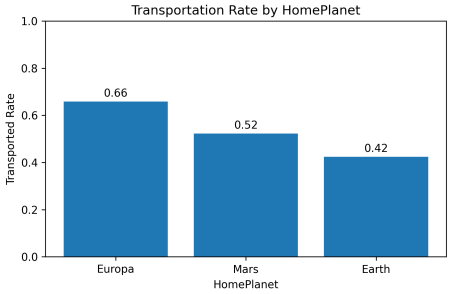

Fig. 2. Transportation rate by HomePlanet.

is not merely a weak categorical variable. It is a central behavioral signal.

This finding also affects how spending should be interpreted. Passengers in CryoSleep are likely to have little or no recorded spending. Therefore, spending behavior is related to both passenger activity and travel status. We used this relationship to create features such as NoSpend and Cryo_NoSpend.

D. HomePlanet Effect

HomePlanet also shows a clear relationship with the target. As shown in Fig. 2, Europa has the highest transported rate, Mars is in the middle, and Earth has the lowest transported rate. This suggests that passenger origin is a useful categorical signal. It should not be removed during preprocessing.

This observation also motivated interaction features such as HomePlanet_Deck. A passenger's origin may interact with cabin placement, travel group, or destination. Encoding such interactions can help tree-based models access subgroup patterns more directly.

E. Cabin Deck Effect

The Cabin field is semi-structured. It contains deck, cabin number, and side in a single string. Fig. 3 shows that cabin deck has a visible relationship with transported rate. Decks B and C have relatively high transported rates, while Deck E and Deck T have lower transported rates.

This finding justified extracting Deck, CabinNum, Side, and Deck_Side. If Cabin had been treated as a raw string or discarded because of missing values, useful spatial information would have been lost. The deck-level difference is also consistent with the idea that the event may have affected certain areas of the ship differently.

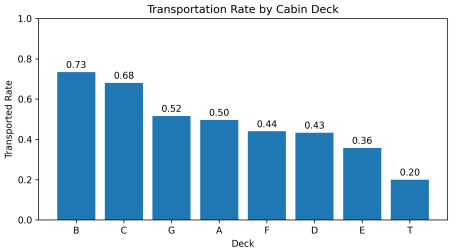

Fig. 3. Transportation rate by cabin deck.

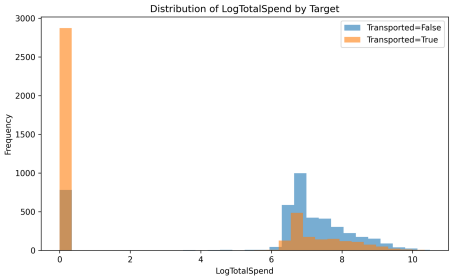

Fig. 4. Distribution of LogTotalSpend by target class.

F. Spending Behavior

The spending columns are sparse and skewed. Fig. 4 shows the distribution of LogTotalSpend by target class. A large number of transported passengers are concentrated in the zero or low-spending region. The nonzero spending range contains both classes, so spending alone cannot determine the target, but it remains a strong behavioral signal.

The EDA led to three spending features. TotalSpend summarizes the overall amount spent across services. NoSpend indicates whether a passenger has no recorded spending. LogTotalSpend reduces skewness and allows the model to split spending values more smoothly. Later feature importance analysis confirmed that NoSpend was the most influential feature in the XGBoost validation model.

G. EDA Summary

Table II summarizes how EDA findings were translated into feature engineering decisions. This table is important because it shows that our feature design followed from observed patterns rather than trial-and-error alone.

V. Data Preprocessing and Feature Engineering

A. Preprocessing Principles

The preprocessing stage had three goals. First, missing values had to be handled without removing too much data. Second, categorical variables had to be represented in a model-compatible way. Third, transformations had to remain reproducible so that the same operations could be applied to both training and test data.

For numerical variables, median imputation was generally preferred because many numerical columns, especially spending columns, are skewed. The median is more robust than the mean under skewed distributions. For categorical variables,

TABLE II
CONNECTION BETWEEN EDA FINDINGS AND FEATURE ENGINEERING

EDA Finding	Feature Decision	Modeling Purpose
CryoSleep has a large transported-rate gap	Preserve CryoSleep; create Cryo_NoSpend	Capture travel-status and spending consistency
HomePlanet differs by target rate	Preserve and encode HomePlanet	Use passenger origin as categorical signal
Cabin deck differs by target rate	Split Cabin into Deck, CabinNum, Side	Capture spatial location information
Spending is sparse and skewed	Create TotalSpend, NoSpend, LogTotalSpend	Represent passenger activity level
PassengerId contains group prefix	Create GroupID, GroupSize, IsSolo	Capture group travel behavior
Name contains surname	Create Surname and SurnameSize	Approximate family-level signal

missing values were either imputed with the most frequent category or handled by models that support categorical features. The exact strategy depended on the model route.

The project also avoided using the target variable during feature construction for the test set. All engineered features were computed from raw columns available in both training and test data. This is important for preventing data leakage and ensuring that the final submission pipeline can be rerun on unseen data.

B. Passenger Group Features

PassengerId has the format of a group identifier followed by an individual number. We split PassengerId into GroupID and GroupMember. We also calculated GroupSize and IsSolo. GroupSize counts how many passengers share the same group prefix, while IsSolo indicates whether the passenger travels alone.

These features are useful because passenger outcomes may not be independent within travel groups. Passengers in the same group may share cabin area, home planet, destination, and cryo sleep behavior. Using PassengerId directly would not help the model, but extracting group structure turns a raw identifier into meaningful information.

C. Cabin Location Features

Cabin was split into Deck, CabinNum, and Side. Deck represents the broad cabin region, CabinNum provides a numerical component, and Side indicates port or starboard side. We also created Deck_Side as an interaction feature.

The reason for creating Deck_Side is that deck and side may not act independently. For example, the transported rate on one side of a deck may differ from the other side. A tree model can sometimes learn this interaction from separate variables, but an explicit interaction feature makes the information easier to access.

D. Spending Features

The raw spending columns were RoomService, FoodCcourt, ShoppingMall, Spa, and VRDeck. We combined them into TotalSpend:

$$\text{TotalSpend} = \sum_{j \in S} x_j, \tag{2}$$

where S is the set of spending columns. We then created NoSpend as

$$\text{NoSpend} = \mathbb{I}(\text{TotalSpend} = 0), \tag{3}$$

and LogTotalSpend as

$$\text{LogTotalSpend} = \log(1 + \text{TotalSpend}). \tag{4}$$

TotalSpend provides a simple summary of passenger activity. NoSpend captures a discrete behavior pattern that is strongly connected with CryoSleep. LogTotalSpend reduces the influence of extreme spending values and makes the distribution easier for tree-based splits.

E. Name and Family Features

The Name field was used to extract Surname. We then calculated SurnameSize, which counts how many passengers share the same surname. The purpose was to approximate family-level relationships. This is different from PassengerId groups because people with the same surname may not always share the same group prefix, and passengers in the same group may not always have the same surname.

Surname features were treated carefully because names can be noisy and may contain missing values. We did not overemphasize them as primary predictors. Instead, they were used as supporting signals that could improve performance when combined with group, cabin, and spending features.

F. Interaction Features

Interaction features were designed to represent combined effects that are difficult to capture with individual raw columns. The main interaction features included Cryo_NoSpend, HomePlanet_Deck, and Deck_Side. These features were chosen because EDA suggested that passenger behavior varies across subgroups.

Cryo_NoSpend combines cryo sleep status with spending behavior. HomePlanet_Deck combines passenger origin with cabin location. Deck_Side combines two parts of cabin placement. These interactions help the model distinguish passengers who look similar under one variable but differ under a combination of variables.

G. Feature Engineering Summary

Table III lists the main engineered feature groups. This table also helps connect the methodology section to the later feature importance analysis.

TABLE III
ENGINEERED FEATURE GROUPS USED IN THE PROJECT

Feature Group	Engineered Features	Source Columns	Purpose
Passenger group	GroupID, GroupMember, GroupSize, IsSolo	PassengerId	Capture group travel structure and solo travelers
Cabin location	Deck, CabinNum, Side, Deck_Side	Cabin	Capture spatial patterns on the ship
Spending behavior	TotalSpend, NoSpend, LogTotalSpend	RoomService, FoodCourt, ShoppingMall, Spa, VRDeck	Represent activity level and spending consistency
Family signal	Surname, SurnameSize	Name	Approximate family-level relationships
Interaction features	Cryo_NoSpend, HomePlanet_Deck, Deck_Side	CryoSleep, NoSpend, HomePlanet, Deck, Side	Capture subgroup-level combined effects

VI. MODEL METHODOLOGY

A. Overall Workflow

The full workflow is shown in Fig. 5. The project started from raw Kaggle data and EDA, then moved to feature engineering, model-specific preprocessing, five model routes, cross-validation and public-score comparison, final model selection, and Kaggle submission. This organization was important because the project was not just a one-model training task. It required a clear chain from data understanding to final reproducible outputs.

A key principle was that model comparison should be based on complete routes rather than algorithms alone. In practice, a model route includes feature selection, imputation, encoding, model training, validation, thresholding, and submission generation. Comparing only algorithm names would hide these important differences. Therefore, when we say "CatBoost route" or "XGBoost with KMeansSMOTE route," we refer to the full pipeline used to produce predictions. The five routes had different roles: HGBC served as a stable scikit-learn baseline, ExtraTrees provided a randomized tree-ensemble comparison, LightGBM provided a fast boosting benchmark, CatBoost tested categorical-feature preservation, and XGBoost with KMeansSMOTE became the final selected route.

B. CatBoost Route

CatBoost was implemented as a strong experimental route because the dataset contains many categorical variables. CatBoost can handle categorical features more naturally than models that require all categories to be one-hot encoded. This made it a good fit for engineered features such as HomePlanet, Destination, Deck, Side, Surname, Deck_Side, and HomePlanet_Deck.

The CatBoost route went through at least two important stages. The early CatBoost model achieved a public Kaggle score of 0.80710. After richer feature engineering and better handling of categorical signals, the improved CatBoost route reached 0.80967. This improvement confirmed that the feature engineering process was moving in a useful direction.

CatBoost was not considered a failed route. It provided evidence that categorical information mattered and helped guide the design of later features. However, the final selected model was not CatBoost because the XGBoost with KMeansSMOTE route achieved a stronger reproducible public score. In the final report, CatBoost should therefore be described as a strong comparison route rather than an abandoned attempt.

C. XGBoost with KMeansSMOTE Route

XGBoost with KMeansSMOTE was the final selected route. The pipeline used compact engineered features, one-hot or encoded categorical input, KMeansSMOTE local resampling, and XGBoost classification. The feature set emphasized CryoSleep, spending consistency, passenger group structure, cabin location, and interaction features.

The motivation for KMeansSMOTE came from the subgroup nature of the data. The target distribution is balanced globally, but EDA showed that transported rate differs substantially by CryoSleep, HomePlanet, and cabin deck. This suggests that the dataset contains local regions with different patterns. KMeansSMOTE helps by clustering the feature space and generating synthetic samples in suitable local regions. This is more targeted than simple random oversampling.

XGBoost was suitable for this route because it can capture nonlinear relationships among engineered features. It can learn interactions between encoded categorical indicators, spending features, and group features through tree splits. Regularization and shrinkage also help reduce overfitting. This route achieved the final public score of 0.81295 and generated the final submission reliably.

D. LightGBM Route

LightGBM was implemented as a fast boosting benchmark. It used engineered features, missing value handling, encoding, validation folds, out-of-fold probability prediction, threshold search, and test probability averaging. The route achieved a public score of 0.80734.

The value of LightGBM was not only its score. It provided a fast way to test whether the engineered features were useful under a different gradient boosting framework. Its score was competitive with CatBoost and above HGBC and ExtraTrees. This supported the conclusion that the feature set was generally useful, not only tailored to one algorithm.

E. ExtraTrees Route

ExtraTrees was implemented as an additional tree-ensemble benchmark. It used median imputation for numerical features, most-frequent imputation for categorical features, one-hot encoding, cross-validation-style validation, and probability averaging. The route achieved a public score of 0.80313.

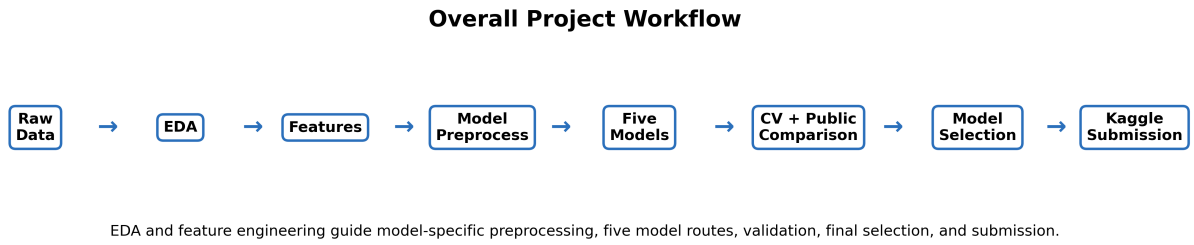

Fig. 5. Overall project workflow from raw data to final Kaggle submission.

ExtraTrees builds many randomized trees independently, which can reduce variance but does not sequentially correct errors in the same way as boosting. Its lower score compared with XGBoost, CatBoost, and LightGBM suggests that boosting methods were better suited to the feature interactions in this dataset. Nevertheless, ExtraTrees was useful because it provided a different ensemble baseline and helped avoid relying on only boosting models.

F. HistGradientBoostingClassifier Route

HGBC was implemented as a reproducible scikit-learn baseline. It used a custom feature builder, SimpleImputer, OrdinalEncoder, and HistGradientBoostingClassifier. It achieved a public score of 0.80266.

The purpose of the HGBC route was not to achieve the highest score. It served as a stable baseline that could be rerun easily. This is important in a course project because a model with simple dependencies and clear preprocessing helps verify whether the feature engineering has value even before more complex model routes are used.

G. Model-Specific Preprocessing

Table IV summarizes the preprocessing strategy and role of each model route. The table shows that the project compared full pipelines rather than isolated algorithms.

H. Training and Validation Procedure

The training process followed a consistent structure across routes. First, engineered features were created from the raw data. Second, the data were split or cross-validated depending on the model route. Third, preprocessing was fitted on the training portion and applied to validation or test data. Fourth, the model was trained and probability predictions were generated. Finally, probabilities were converted into Boolean labels and formatted for Kaggle submission.

Threshold selection was considered during experimentation because accuracy depends on the final binary decision. However, the final decision did not depend only on a single threshold-tuned validation result. Public leaderboard score and rerun reliability were more important. A model that is too sensitive to one split or one threshold would be risky for final submission.

I. Hyperparameter Selection

Hyperparameter tuning was performed pragmatically. Because the project had course deadlines and limited computing resources, an exhaustive search over all possible combinations was not realistic. We focused on parameters that have clear influence on tree-based models: number of estimators, learning rate, maximum depth, subsampling ratio, column sampling ratio, and regularization.

For XGBoost, the final route used a moderate tree depth and a relatively small learning rate to balance learning capacity and overfitting risk. For LightGBM, the focus was fast iteration and fold-based probability averaging. For CatBoost, the focus was preserving categorical structure and improving feature design. For ExtraTrees and HGBC, the emphasis was on comparison value and reproducibility rather than extensive tuning.

J. Model Selection Criteria

The final model was selected using multiple criteria, not score alone. Table V summarizes the criteria. Public score was important because it is the official Kaggle metric. Reproducibility was also essential because the final code must be submitted and rerun. Pipeline clarity mattered because the report and presentation needed to explain the method. Model suitability and practical cost were considered because a complicated route that is hard to run may not be suitable for final submission.

VII. Experimental Study and Results Analysis

A. Experimental Setup

The primary evaluation metric was Kaggle public accuracy, which follows the competition setting. Each model route generated a test prediction file, and the public score was recorded after submission. Local validation diagnostics were also used to analyze model behavior. These diagnostics included confusion matrix, ROC curve, and feature importance.

It is important to distinguish between Kaggle public score and local validation results. Kaggle public score is computed on hidden labels by the competition platform. Local validation results are computed on a held-out split from the training data. The two forms of evaluation answer different questions. Kaggle score asks how well the submitted predictions perform on the public hidden split. Validation diagnostics ask how the model behaves on known labels and whether the behavior is reasonable.

TABLE IV
SUMMARY OF MODEL-SPECIFIC PREPROCESSING AND ROLES

Model Route	Categorical Strategy	Main Preprocessing Focus	Role in the Project
CatBoost	Native categorical handling	Preserve categorical-heavy engineered features and allow CatBoost to process category signals directly	Strong experimental route and categorical-feature evidence
XGBoost + KMeansSMOTE	One-hot or encoded features	Compact engineered features, numerical matrix construction, local resampling, XGBoost training	Final selected reproducible model
LightGBM	One-hot or encoded features	Rich engineered features, validation folds, threshold search, averaged test probabilities	Fast boosting benchmark
ExtraTrees	One-hot encoded features	Median/mode imputation, randomized tree ensemble, probability averaging	Additional ensemble comparison
HGBC	Ordinal encoding	SimpleImputer, OrdinalEncoder, scikit-learn style reproducible pipeline	Stable baseline model

TABLE V
FINAL MODEL SELECTION CRITERIA

Criterion	Meaning in This Project
Public score	Kaggle public leaderboard accuracy
Reproducibility	Ability to rerun code and regenerate submission files
Pipeline clarity	Whether feature engineering and preprocessing steps are understandable
Model suitability	Whether the algorithm matches the engineered feature representation
Practical cost	Training time, dependencies, and deadline risk

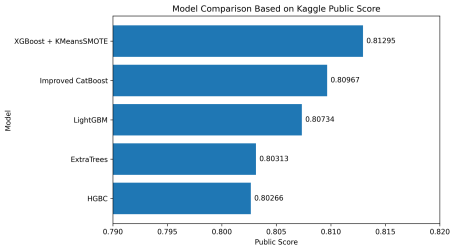

Fig. 6. Model comparison based on Kaggle public score.

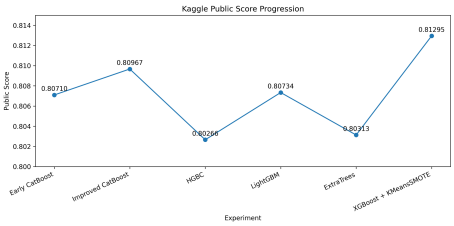

Fig. 7. Kaggle public score progression across model routes.

B. K-Fold Validation Results of the XGBoost Route

In addition to the public leaderboard score, we recorded k-fold validation results for the XGBoost experiment under different random seeds and resampling strategies. These scores are not Kaggle public scores. They are local cross-validation accuracies used to compare the stability of the XGBoost route before choosing submission files. Table VI shows the recorded 10-fold validation results.

The k-fold results show that the improvement from resampling is not uniform across every seed. This is expected because the target is not severely imbalanced at the global level. However, the KMeans-style route produced the strongest local CV result in two of the four seeds, including the highest observed local mean of 0.81251. This supported the decision to keep the KMeansSMOTE route as a serious final candidate, while still using the Kaggle public score as the official comparison metric.

C. Overall Model Comparison

Table VII shows the Kaggle public score of each model route. XGBoost with KMeansSMOTE achieved the highest score of 0.81295. Improved CatBoost achieved 0.80967, LightGBM achieved 0.80734, ExtraTrees achieved 0.80313, and HGBC achieved 0.80266.

Fig. 6 visualizes the same comparison. The score differences are not extremely large, but they are meaningful in a Kaggle competition where many teams are separated by small margins. The final model improved over the early CatBoost score by 0.00585.

The comparison supports two points. First, boosting-based models generally performed better than the simpler baseline and randomized tree ensemble. Second, model choice alone does not explain the result. The preprocessing and feature representation behind each model were also important.

D. Score Progression

Fig. 7 shows the public score progression across major experiments. The project did not improve in a straight line. Some intermediate routes, such as HGBC and ExtraTrees, scored lower than earlier CatBoost attempts. However, these lower scores were still useful because they provided comparison evidence.

The progression shows a realistic modeling process. Early CatBoost gave a strong start. Improved CatBoost showed that feature engineering improved performance. HGBC provided a reproducible baseline. LightGBM tested a fast boosting alternative. ExtraTrees tested a different ensemble family. XGBoost with KMeansSMOTE produced the best final reproducible result. In this sense, the intermediate routes were not wasted even if they did not become the final submission model.

E. Per-Model Interpretation

The CatBoost route performed strongly because it matched the categorical structure of the data. Its score of 0.80967 was

TABLE VI
LOCAL 10-FOLD CROSS-VALIDATION RESULTS FOR XGBOOST EXPERIMENTS

Seed	Resampling	CV Mean	CV Std.	Training Shape	Interpretation
42	None	0.80904	0.01349	8693×15	Strong non-resampled baseline
42	SMOTE	0.80996	0.01214	8756×15	Slight local improvement
42	KMeans-style	0.80874	0.01179	8763×15	Stable but not best for this seed
123	None	0.80709	0.01191	8693×15	Lower baseline under new seed
123	SMOTE	0.80528	0.01102	8756×15	Resampling did not help here
123	KMeans-style	0.81068	0.01179	8763×15	Best method for this seed
2023	None	0.80674	0.00990	8693×15	Stable but lower mean
2023	SMOTE	0.80733	0.00792	8756×15	Lower variance
2023	KMeans-style	0.80954	0.01365	8763×15	Competitive mean
2140	None	0.80778	0.01119	8693×15	Stable baseline
2140	SMOTE	0.80687	0.01125	8756×15	No clear gain
2140	KMeans-style	0.81251	0.01106	8763×15	Highest local CV mean

TABLE VII
MODEL COMPARISON BASED ON KAGGLE PUBLIC SCORE

Rank	Model	Public Score
1	XGBoost + KMeansSMOTE	0.81295
2	Improved CatBoost	0.80967
3	LightGBM	0.80734
4	ExtraTrees	0.80313
5	HGBC	0.80266

the second best result. This confirms that categorical signals such as HomePlanet, Deck, Side, and interaction categories were useful. CatBoost's result also supports the decision to preserve categorical information during feature engineering.

The XGBoost with KMeansSMOTE route performed best because it combined compact features, encoded inputs, local resampling, and a strong nonlinear classifier. Its advantage was not only the XGBoost algorithm itself. It was the combination of feature representation and resampling that made the route effective.

LightGBM achieved a competitive score and served as a fast benchmark. Its performance showed that boosting methods could make good use of the engineered features. ExtraTrees and HGBC scored lower, but they still contributed to the analysis. ExtraTrees showed that independent randomized tree ensembles were less effective than boosting in this setting. HGBC gave a stable baseline and helped estimate the value of more complex routes.

F. Computational Cost and Practical Suitability

All five models were trainable on a normal laptop environment because the dataset is moderate in size. However, the models differed in iteration speed, dependency complexity, and pipeline risk. HGBC was simple and easy to rerun. LightGBM was fast and suitable for repeated experiments. CatBoost was strong but depended on correct handling of categorical columns. XGBoost required more preprocessing, but after the encoded feature matrix was created, it provided strong and reliable predictions. ExtraTrees was easy to run but less competitive.

From a final submission perspective, reliability mattered. The final model needed to generate a clean CSV file with PassengerId and Transported predictions. A route with many manual steps would increase deadline risk. Therefore, the selected route had to be not only accurate but also organized and reproducible.

G. Why XGBoost with KMeansSMOTE Was Selected

The final decision can be explained from four angles. First, XGBoost with KMeansSMOTE achieved the highest reproducible public score. Second, the compact feature set matched XGBoost well after encoding. Third, KMeansSMOTE was consistent with subgroup patterns observed in EDA. Fourth, the route was clear enough to document and rerun.

The dataset contains multiple subgroup effects. CryoSleep strongly changes transported rate. HomePlanet categories differ. Cabin deck differs. Spending behavior interacts with CryoSleep. These patterns suggest that one global linear rule is insufficient. KMeansSMOTE helps represent local regions before XGBoost learns nonlinear decision boundaries.

This does not mean CatBoost was weak. CatBoost remained a strong route and helped validate the categorical feature design. The final conclusion is more specific: under our feature design, preprocessing constraints, and reproducibility requirement, XGBoost with KMeansSMOTE was the best final route.

H. Validation Confusion Matrix

A held-out validation experiment was conducted to analyze model behavior. The validation model achieved an accuracy of approximately 0.8148. Fig. 8 shows the confusion matrix. The model correctly classified 696 non-transported passengers and 721 transported passengers. It misclassified 167 non-transported passengers as transported and 155 transported passengers as non-transported.

The false positive and false negative counts are close. This indicates that the model does not heavily favor one class. This is consistent with the balanced target distribution and supports the use of accuracy as the primary metric. The remaining errors likely come from ambiguous passengers whose features overlap across classes, such as passengers with partial spending, missing cabin information, or unusual subgroup combinations.

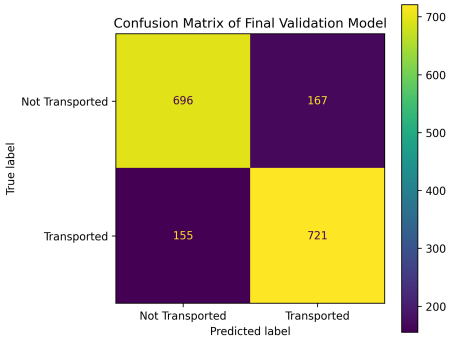

Fig. 8. Confusion matrix on the held-out validation split.

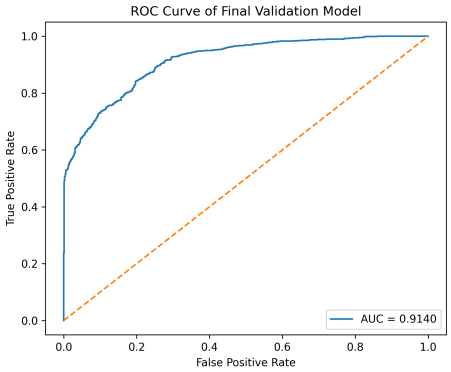

Fig. 9. ROC curve on the held-out validation split.

I. ROC Curve

Fig. 9 shows the ROC curve of the validation model. The AUC is approximately 0.9140, indicating strong ranking ability across classification thresholds.

The ROC curve provides information beyond accuracy. Accuracy depends on a fixed decision threshold, while AUC evaluates whether the model tends to assign higher probabilities to positive examples than to negative examples. The high AUC suggests that the probability outputs are meaningful and that threshold tuning may provide additional improvement.

J. Feature Importance

Fig. 10 shows the top feature importances from the XGBoost validation model. NoSpend is the most important feature by a large margin. CryoSleep-related features, TotalSpend, HomePlanet, Deck, and interaction features also appear among the top features.

This result matches the EDA findings. CryoSleep and spending behavior were already visible as strong signals. The feature importance analysis confirms that the model uses these signals heavily. The presence of HomePlanet_Deck and Deck_Side also supports the value of interaction features. The model benefited from engineered combinations rather than relying only on raw columns.

K. Feature Contribution and Ablation-Oriented Analysis

A full retraining-based ablation study for every feature group was not completed before the final deadline. Therefore,

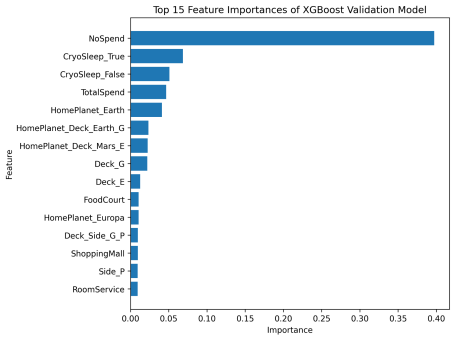

Fig. 10. Top 15 feature importances of the XGBoost validation model.

we do not report fabricated score drops for removed features. Instead, we performed an ablation-oriented contribution analysis by connecting three sources of evidence: EDA findings, feature importance, and model behavior. This analysis is weaker than a full ablation experiment, but it still helps identify which feature groups were most likely to contribute to the final model. Table VIII summarizes the evidence.

This analysis also explains why future work should include a formal ablation study. The strongest candidate for ablation would be the spending feature group, followed by CryoSleep-related and cabin-related features. A proper ablation would retrain the same model under the same folds after removing one feature group at a time.

L. Error Analysis

The remaining errors can be understood from the structure of the data. Some passengers have similar spending behavior but different target outcomes. Others have missing values in important fields such as CryoSleep, Cabin, or HomePlanet. Some passengers may belong to groups where the majority pattern differs from individual behavior. These cases are difficult because the model must decide between competing signals.

False positives are passengers predicted as transported but actually not transported. Many such cases may have features similar to transported passengers, such as no spending, CryoSleep-like behavior, or cabin decks with high transported rates. False negatives are passengers predicted as not transported but actually transported. These may include passengers whose spending behavior or cabin location resembles the negative class even though they were transported.

This error pattern supports the idea that the task cannot be solved by a few simple rules. A high CryoSleep transported rate does not mean every CryoSleep passenger is transported. A low spending amount does not guarantee the target. A model must combine multiple signals and handle ambiguous regions.

M. Comparison with CatBoost

The CatBoost route deserves special discussion because it was the second strongest model. Its performance showed that preserving categorical features can be valuable. CatBoost naturally handles categorical variables, so it could use features

TABLE VIII
ABLATION-ORIENTED FEATURE CONTRIBUTION ANALYSIS

Feature Group	Evidence from EDA or Importance	Expected Effect if Removed	Reasoning
Spending features	NoSpend is the top feature; TotalSpend is also important	Large decrease expected	Spending is strongly connected with CryoSleep and passenger activity
CryoSleep features	CryoSleep groups show a large transported-rate gap	Large decrease expected	CryoSleep separates high-rate and low-rate passenger groups
Cabin features	Deck transport rates differ; Deck and Side features appear in importance list	Moderate decrease expected	Cabin location contains spatial structure
HomePlanet features	Europa, Mars, and Earth have different transported rates	Moderate decrease expected	Passenger origin is a meaningful categorical signal
Interaction features	HomePlanet_Deck and Deck_Side appear in top features	Moderate decrease expected	Interactions make subgroup patterns easier for trees to split
Group/family features	PassengerId and surname contain group structure	Small to moderate decrease expected	These features help where travel group behavior is consistent

such as HomePlanet, Deck, Side, Surname, and interaction categories without forcing them into a fully sparse representation.

However, the final XGBoost route performed better in the public leaderboard. This indicates that the compact encoded feature representation, combined with KMeansSMOTE and XGBoost, matched the final submission objective more closely. The conclusion should not be that CatBoost was poor. Instead, CatBoost was an important comparison route that confirmed the value of categorical engineering, while XGBoost with KMeansSMOTE became the final model because it achieved the strongest reproducible score.

N. Reproducibility Analysis

Reproducibility was an explicit selection criterion. A course project needs code that can be inspected, rerun, and connected to the final reported result. If a route produces a high score but depends on undocumented manual changes, it is risky. The final route had a clearer sequence: load data, build features, preprocess, apply KMeansSMOTE, train XGBoost, predict test labels, and export the submission file.

The report also distinguishes between official Kaggle score and local validation diagnostics. This avoids overstating validation results. The official score is 0.81295. The validation accuracy and AUC are supporting evidence, not replacements for the Kaggle evaluation.

O. Result Summary

Table IX summarizes the main conclusions from the experimental section. The table is included to make the logic clearer and to connect results with interpretation.

VIII. FINAL KAGGLE RANKING AND SUBMISSION MATERIALS

The final selected model was XGBoost with KMeansSMOTE. The final public Kaggle score was 0.81295, and the submission was made under the team account milkloong@MLW. The final ZIP package should contain the final report, source code, presentation slides, GitHub repository link, final submission file, and screenshots of the Kaggle submission logs.

The final leaderboard ranking is not reported because the ranking information was not available at the time of final report preparation. We therefore report only the verified public Kaggle score and do not invent a rank.

The GitHub repository for this project is available at:

`https://github.com/camellia1012-Yitao/Spaceship-Titanic-MLW.git`

The repository contains the Python source files for all five model routes, a final XGBoost experiment notebook, generated submission files, figures, requirements, and a README explaining how to place the Kaggle CSV files and rerun the workflow. Kaggle submission log screenshots should be included in the submitted ZIP file and, if available, placed in the repository under `docs/`.

IX. LIMITATIONS

A. Public Leaderboard Uncertainty

The Kaggle public score is useful but limited. It is computed on only part of the hidden test set. A model with a slightly higher public score does not always perform better on the private leaderboard. For this reason, we did not select the final model based only on a one-time public score. We also considered reproducibility and model clarity.

B. Validation Split Dependence

The confusion matrix, ROC curve, and feature importance were generated from a held-out validation split. A single split can be affected by randomness. Repeated cross-validation would provide a more stable estimate. This limitation does not invalidate the analysis, but it means validation diagnostics should be interpreted as supporting evidence rather than exact final performance.

C. Feature Importance Interpretation

Feature importance from tree-based models should not be interpreted as causal evidence. NoSpend is highly important, but it is correlated with CryoSleep and other behavior features. Similarly, HomePlanet and Deck interactions may represent underlying subgroup patterns rather than direct causes. Therefore, feature importance should be read together with EDA and domain reasoning.

TABLE IX
SUMMARY OF EXPERIMENTAL FINDINGS

Result Component	Observation	Interpretation
Model comparison	XGBoost + KMeansSMOTE achieved 0.81295	Best reproducible public score among tested routes
Score progression	Scores did not increase linearly	Intermediate models provided useful comparison evidence
Confusion matrix	False positives and false negatives were close	Model behavior was balanced across two classes
ROC curve	AUC was approximately 0.9140	Probability ranking was strong on validation split
Feature importance	NoSpend, CryoSleep, TotalSpend, HomePlanet, and cabin interactions were important	EDA-driven features aligned with model behavior
CatBoost comparison	CatBoost was second strongest	Categorical signals were valuable but final XGBoost route performed better

D. KMeansSMOTE Ablation

The final route used KMeansSMOTE, but a more rigorous ablation would compare XGBoost with and without KMeansSMOTE under the same validation folds and threshold strategy. This would isolate the effect of resampling more clearly. In the current project, KMeansSMOTE is justified by final public score and subgroup reasoning, but future work should test its independent contribution.

E. Feature Engineering Ceiling

The final score suggests that the models captured many strong patterns, but some information may still be missing. For example, group-level aggregation and cabin-neighborhood features could be further developed. The performance ceiling may be limited more by feature representation than by model complexity.

X. FUTURE WORK

First, feature engineering could be refined. More advanced group-level features could summarize the behavior of passengers in the same group. Family-level aggregation could use surname information more carefully. Cabin-neighborhood features could use CabinNum to estimate local spatial regions instead of treating deck and side alone.

Second, a stronger ensemble could be built from CatBoost and XGBoost. Since CatBoost and XGBoost use different preprocessing assumptions, their errors may not be identical. A carefully validated blend of their probability outputs could improve robustness. However, such an ensemble should be evaluated with cross-validation to avoid overfitting to the public leaderboard.

Third, threshold optimization could be made more systematic. The final prediction depends on converting probabilities into Boolean labels. A threshold selected from cross-validation may improve accuracy compared with a fixed threshold of 0.5.

Fourth, repeated cross-validation should be used to reduce split randomness. This would provide better estimates of model stability and help compare routes more fairly.

Fifth, the final codebase can be improved by adding a clearer command-line workflow. For example, one script could train the model, another could generate figures, and another could create the submission file. This would make the project easier to reproduce and easier to grade.

XI. CONCLUSION

This project developed a complete machine learning workflow for the Spaceship Titanic classification task. We performed EDA, transformed raw fields into structured features, implemented five model routes, compared their public Kaggle scores, and selected the final model based on both performance and reproducibility.

The final XGBoost with KMeansSMOTE pipeline achieved a public Kaggle score of 0.81295. The analysis showed that CryoSleep, spending behavior, cabin location, passenger group structure, and HomePlanet were important signals. Feature importance confirmed that NoSpend and CryoSleep-related features were especially influential. The comparison across five models showed that the final result came from the whole pipeline rather than from a single algorithm.

The most important lesson is that tabular machine learning depends heavily on representation. A strong model cannot fully compensate for weak features, and a good feature set must still be paired with suitable preprocessing. In this project, the best result came from combining EDA-driven feature engineering, model-specific preprocessing, local resampling, and a reproducible XGBoost pipeline.

CONTRIBUTION STATEMENT

The team contribution is summarized as follows:

- Lu Kejie (2430034044): ExtraTrees model implementation, comparison analysis, and final presentation support.
- Hou Rui (2430034019): LightGBM benchmark model, validation experiments, and feature engineering support.
- Yang Zhongxing (2430034088): XGBoost with KMeansSMOTE final modeling, submission generation, and comparison experiments.
- Guo Yitao (2430034015): CatBoost route analysis, GitHub organization, report integration, and presentation integration.
- Chen Ziheng (2430034008): HGBC baseline, supporting analysis, and challenge discussion.
- Whole team: EDA discussion, feature engineering design, model comparison, final review, and presentation preparation.

APPENDIX A
KAGGLE SUBMISSION LOG SCREENSHOTS

Fig. 11 shows the Kaggle submission log for the Spaceship Titanic competition. The screenshot records the submitted files

and public scores for the five model routes. The final selected submission is the XGBoost + KMeansSMOTE file with a public score of 0.81295, which is the highest score among our submitted models.

APPENDIX B
GITHUB AND CODE REPRODUCIBILITY NOTES

The GitHub repository for the project is:

`https://github.com/camellia1012-Yitao/Spaceship-Titanic-MLW.git`

The repository contains a README file explaining how to reproduce the workflow. The README includes environment setup, required packages, data placement, training instructions, and submission generation steps. The Kaggle data files are not uploaded to GitHub; instead, the README states that `train.csv`, `test.csv`, and `sample_submission.csv` should be placed locally before running the code.

The final repository structure is organized as follows:

```
Spaceship-Titanic-MLW/
  README.md
  requirements.txt
  .gitignore
  docs/
  figures/
  notebooks/
    final_xgboost_full_experiment.ipynb
  result/
  src/
    preprocessing.py
    train_CatBoost.py
    train_ExtraTree.py
    train_HGBC.py
    train_LightGBM.py
    train_XGBoost_KMeansSMOTE.py
  submissions/
    final_submission.csv
    model submission files
```

The most important reproducibility requirement is that a future reader should be able to load the data, build engineered features, train the selected model, generate predictions, and create the final submission file without relying on undocumented manual changes.

APPENDIX C
DETAILED FINAL FILE CHECKLIST

Before final submission, the ZIP package should include the following files:

- Final project report PDF.
- LaTeX source file or editable report source if required.
- Runnable source code or notebooks.
- GitHub repository link in the report and README.
- Final presentation slides in PPTX or PDF format.
- Kaggle submission log screenshots.
- Final submission CSV file.
- README file with environment, dependencies, and running instructions.
- requirements.txt or environment description.

APPENDIX D
ADDITIONAL NOTES ON VALIDATION FIGURES

The confusion matrix, ROC curve, and feature importance plots were generated from a held-out validation experiment using the XGBoost-based engineered feature representation. They are included to explain model behavior. The official competition result remains the Kaggle public score of 0.81295 from the final XGBoost with KMeansSMOTE submission.

The validation figure should not be described as a direct Kaggle test-set evaluation because Kaggle test labels are hidden. This distinction is important for academic honesty and technical correctness.

APPENDIX E
EXPANDED METHODOLOGICAL REFLECTION

One of the lessons from the project is that model comparison in applied machine learning is rarely a pure comparison of algorithms. Each model route includes decisions about preprocessing, feature construction, validation, parameter settings, and submission formatting. These decisions can change the final score as much as the model family itself.

For example, CatBoost is powerful when categorical columns are preserved, but if all categorical variables are forced into one-hot encoding, CatBoost may lose part of its advantage. XGBoost is powerful after the data are converted into a numerical matrix, but it depends on careful encoding and feature construction. HGBC is simple and reproducible, but ordinal encoding may not represent unordered categories perfectly. These differences show why the project should be described as a comparison of pipelines rather than a ranking of model names.

Another lesson is that EDA and feature importance should be connected. In this project, EDA suggested that CryoSleep and spending behavior were strong signals. Feature importance later confirmed that NoSpend, CryoSleep, and TotalSpend were among the most important features. This agreement increased confidence in the feature design. If feature importance had highlighted features with no reasonable interpretation, the model would have been harder to defend.

Finally, reproducibility is a practical skill. A final report is stronger when the model can be rerun, the figures can be regenerated, and the submission file can be traced to a specific pipeline. This is why the project emphasized clear code organization, a public GitHub repository, and explicit submission evidence.

APPENDIX F
POTENTIAL ABLATION STUDY DESIGN

A more complete version of the project could include retraining-based ablation studies. An ablation study removes or modifies one component at a time and measures the effect on validation performance. Section VII already provides an ablation-oriented contribution analysis based on EDA and

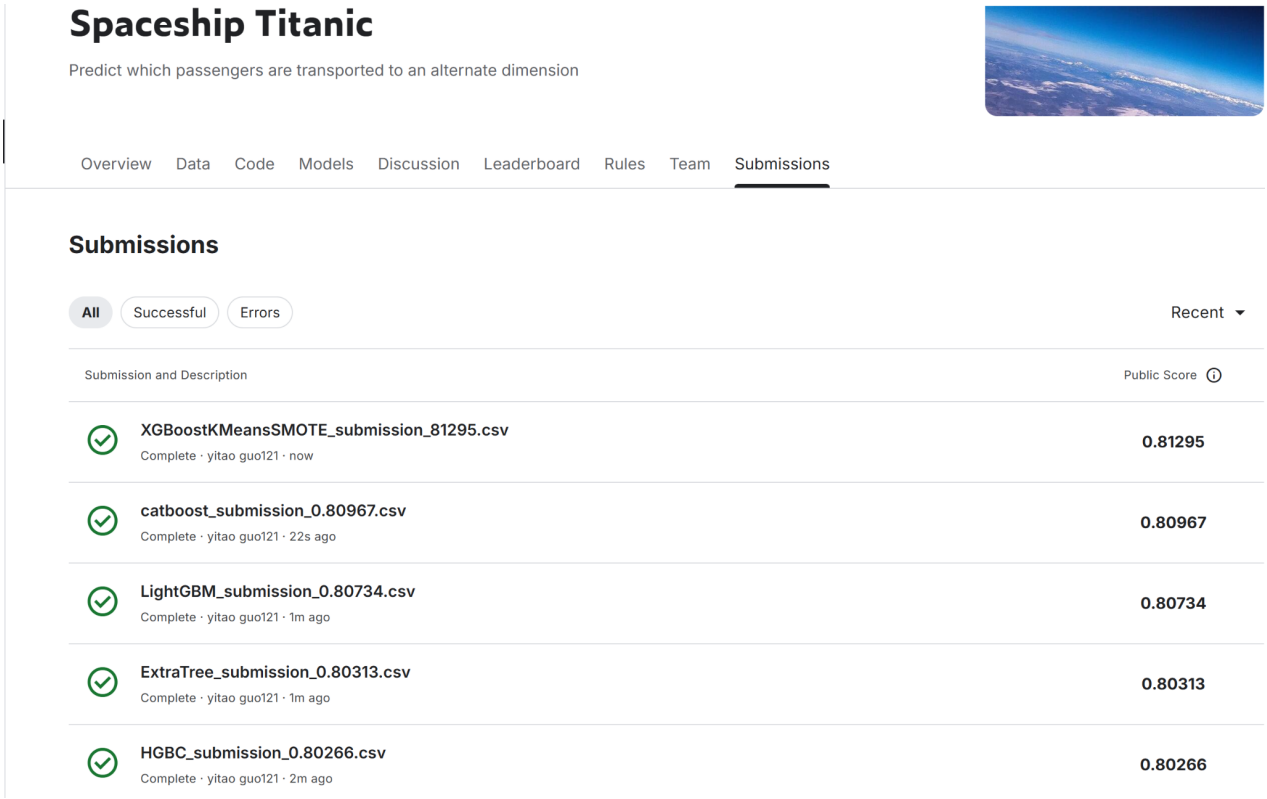

Fig. 11. Kaggle submission log showing the public scores of the submitted model routes.

feature importance. A stricter future ablation design would use fixed folds and the same thresholding rule for every comparison:

- **Remove spending features:** compare the full model with a model without TotalSpend, NoSpend, and LogTotalSpend to measure the contribution of passenger activity signals.
- **Remove cabin features:** compare the full model with a model without Deck, Side, CabinNum, and Deck_Side to test the value of spatial location information.
- **Remove group features:** compare the full model with a model without GroupID, GroupSize, and IsSolo to test the value of travel group structure.
- **Remove interaction features:** compare the full model with a model without Cryo_NoSpend, HomePlanet_Deck, and Deck_Side to measure whether explicit subgroup interactions help.
- **Remove KMeansSMOTE:** compare XGBoost with and without KMeansSMOTE to isolate the contribution of local resampling.
- **CatBoost vs. XGBoost feature setting:** compare similar feature groups under categorical handling and encoded matrix learning assumptions.

Such ablation studies would make the experimental analysis stronger because they would separate the effect of each major design decision. They would also reduce the risk of over-explaining a component that may not independently improve performance.

APPENDIX G
DISCUSSION OF ACADEMIC INTEGRITY AND REPORT WRITING

The final report should avoid overstating results. The Kaggle public score should be reported exactly as obtained. Unknown values should not be invented. In this final version, the GitHub link is provided, while the final leaderboard ranking is not reported because it was not available at the time of report preparation. Validation metrics are described as local validation results, not official Kaggle test results.

The report should also avoid presenting all models as equally important if the experimental evidence shows otherwise. XGBoost with KMeansSMOTE and CatBoost were the strongest routes, but LightGBM, ExtraTrees, and HGBC still belong in the report because the course project requires multiple model implementation and comparison. Their role is to provide baselines, benchmarks, and evidence for final selection.

REFERENCES

[1] Kaggle, "Spaceship Titanic," Kaggle Competition. [Online]. Available: https://www.kaggle.com/competitions/spaceship-titanic
[2] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
[3] P. Geurts, D. Ernst, and L. Wehenkel, "Extremely randomized trees," *Machine Learning*, vol. 63, no. 1, pp. 3–42, 2006.
[4] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
[5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[6] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, 2017, pp. 3146–3154.

[7] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, 2018, pp. 6638–6648.

[8] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[9] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[10] G. Lemaitre, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning," *Journal of Machine Learning Research*, vol. 18, no. 17, pp. 1–5, 2017.

[11] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.

[12] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61.

[13] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[14] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[15] IEEE, "Preparation of articles for IEEE Transactions and Journals," IEEE Author Center template and instructions, 2022.